

C PROGRAMMING DEBUGGING TOOLS

Practical Assignment Report

STUDENT DETAILS

NAME	GURVEER SINGH
------	---------------

SAP ID	590032925
--------	-----------

COURSE	B.Tech CSE - 1st Year
--------	-----------------------

BATCH	15
-------	----

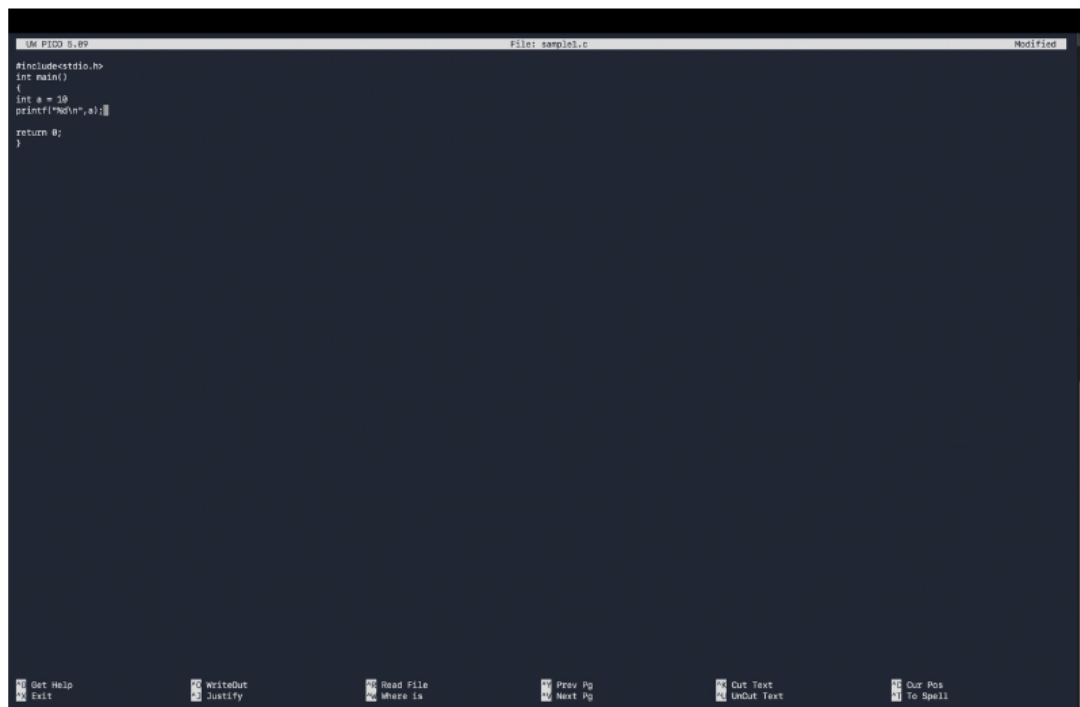
1. Fundamentals of Errors and Debugging

An error is a mistake in source code. A bug is a defect that causes incorrect or unexpected behavior. Debugging is the systematic process of finding, analyzing, and removing such defects.

Error Type	Meaning	Typical Example
Syntax	Violates C grammar	Missing semicolon
Compilation/Type	Invalid type or declaration	Incompatible assignment
Linker	Required definition cannot be linked	Undefined function
Runtime	Fails during execution	Division by zero / invalid memory
Logical	Runs but gives wrong result	Incorrect loop condition

Case 1: Syntax & Compilation Error

A missing semicolon prevents successful compilation. Clang identifies the source line and reports that a semicolon is expected.

A screenshot of a code editor window titled "vim PID: 8,89" and "File: sample.c". The code inside is:

```
#include <stdio.h>
int main()
{
    int a = 10;
    printf("%d\n", a);
    return 0;
}
```

The code is missing a semicolon at the end of the line `printf("%d\n", a);`. The editor has a dark theme and a status bar at the bottom with various icons and text like "Get Help", "WriteOut", "Read File", "Prev Pg", "Cut Text", "Out Pos", "Exit", "Justify", "Where is", "Next Pg", "UnCut Text", and "To Spell".

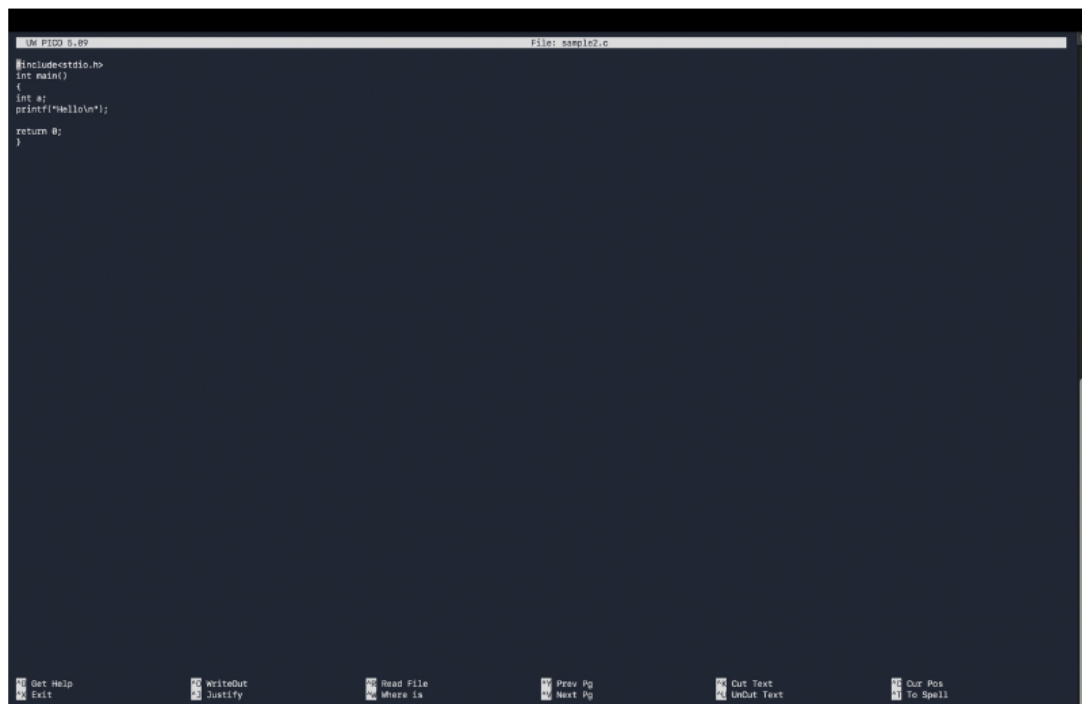
Source code containing the syntax error.

```
dhruvpandey@DHRLV9-MacBook-Air ~ % clang sample1.c -o sample1.c
sample1.c:4:11: error: expected ';' at end of declaration
    4 | int a = 10;
      |           ^
1 error generated.
dhruvpandey@DHRLV9-MacBook-Air ~ %
```

Terminal output showing the Clang syntax-error diagnostic.

Case 2: Compiler Warnings (-Wall, -Wextra)

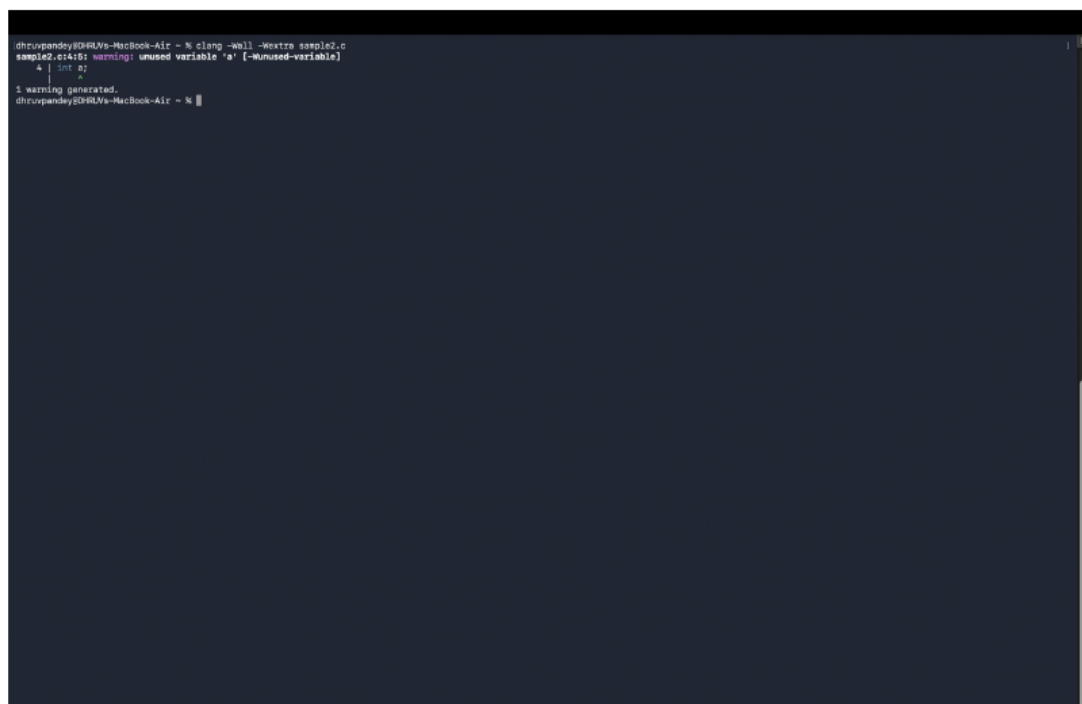
Warnings identify suspicious code that is still legal C. Here, variable `a` is declared but never used.



```
File: sample2.c
#include<stdio.h>
int main()
{
    int a;
    printf("Hello\n");
    return 0;
}
```

The screenshot shows a code editor window titled "File: sample2.c". The code inside is a simple C program that includes `<stdio.h>`, has a `main` function, declares an integer variable `a`, prints "Hello\n", and returns 0. The variable `a` is declared but never used. At the bottom of the editor, there is a toolbar with icons for "Get Help", "Exit", "WriteOut", "Justify", "Read File", "Where Is", "Prev Pg", "Next Pg", "Cut Text", "UnCut Text", and "Our Pos To Spell".

Program containing an unused variable.



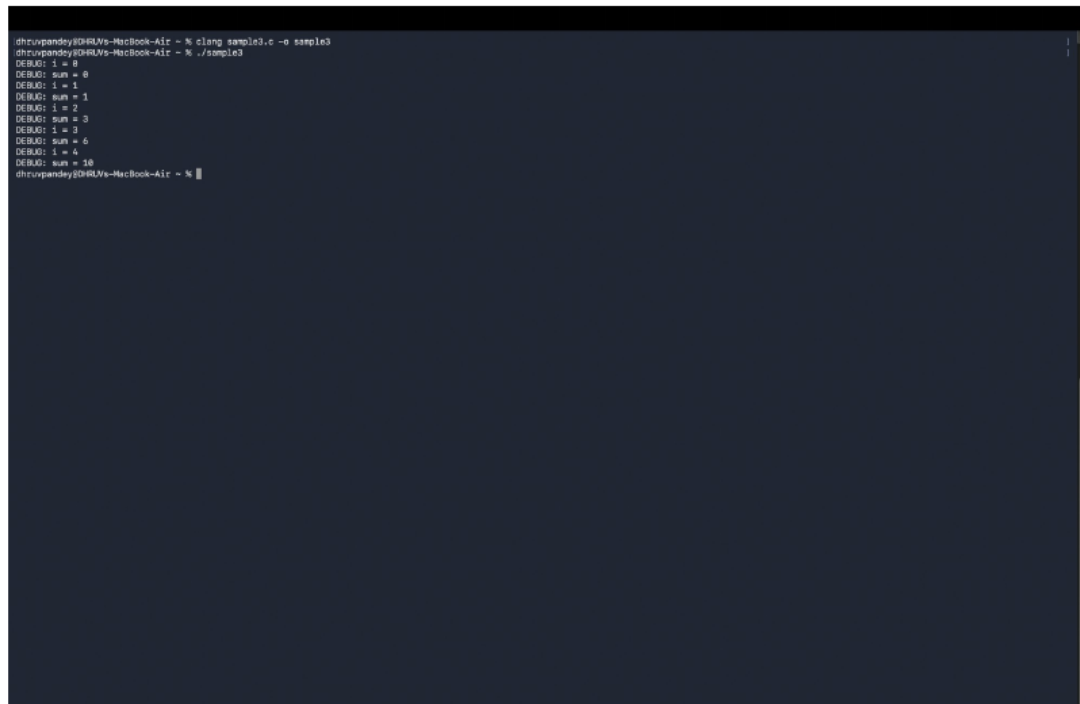
```
dhruvpendey@DHRLVa-MacBook-Air ~ % clang -Wall -Wextra sample2.c
sample2.c:4:15: warning: unused variable 'a' [-Wunused-variable]
    4 |     int a;
      |               ^
1 warning generated.
dhruvpendey@DHRLVa-MacBook-Air ~ %
```

The screenshot shows a terminal window where the command `clang -Wall -Wextra sample2.c` has been executed. The output shows a warning: `sample2.c:4:15: warning: unused variable 'a' [-Wunused-variable]`, pointing to the declaration of `int a;`. Below the warning, it says "1 warning generated." and the prompt `dhruvpendey@DHRLVa-MacBook-Air ~ %` is shown.

Terminal warning generated with -Wall and -Wextra.

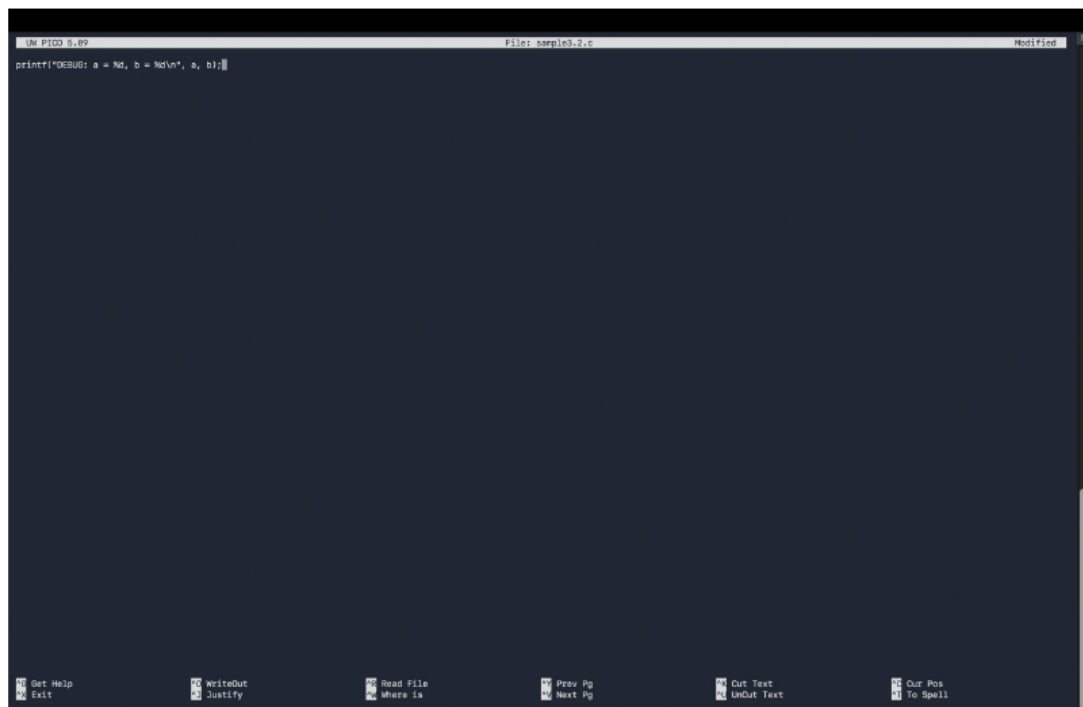
Case 3: printf-Based Debugging (Logical Bug)

Debug print statements expose the values of `i` and `sum` on each loop iteration. The trace shows the program reaches a final sum of 10.

A terminal window with a dark background showing the output of a C program. The program uses printf statements to debug a loop. The output shows the values of 'i' and 'sum' at each iteration. The loop runs for i values from 0 to 4, and the final sum is 10.

```
dhruvpandey@DHRLVs-MacBook-Air ~ % clang sample3.c -o sample3
dhruvpandey@DHRLVs-MacBook-Air ~ % ./sample3
DEBUG: i = 0
DEBUG: sum = 0
DEBUG: i = 1
DEBUG: sum = 1
DEBUG: i = 2
DEBUG: sum = 3
DEBUG: i = 3
DEBUG: sum = 6
DEBUG: i = 4
DEBUG: sum = 10
dhruvpandey@DHRLVs-MacBook-Air ~ %
```

Terminal trace produced by printf-based debugging.



The screenshot shows a window titled "PICO 5.89" with a menu bar (File, Edit, View, Window, Help) and a toolbar. The main text area contains the following C code:

```
File: sample3.2.c  
printf("DEBUG: a = %d, b = %d\n", a, b);
```

The status bar at the bottom displays several icons and labels: "Get Help", "Exit", "WriteOut", "Justify", "Read File", "Where Is", "Prev Pg", "Next Pg", "Cut Text", "UnCut Text", "Our Pos", and "To Spell".

Example of a debug printf statement.

Case 4: Debugging a Logical Error using GDB

The loop uses $i < 5$, so 5 is never included. GDB can stop on the sum statement and display i and sum at each iteration. The actual result is 10; the intended result is 15.

The first screenshot shows the GDB online debugger interface. The code being debugged is a C program that calculates the sum of integers from 1 to 5. The code is as follows:

```
1 #include <stdio.h> // Line 1
2 int main() // Line 2
3 { // Line 3
4     int i, sum = 0; // Line 4
5     for(i = 1; i < 5; i++) // Line 5 (Logical Bug: should be i <= 5)
6     { // Line 6
7         sum = sum + i; // Line 7
8     } // Line 8
9     printf("Sum = %d\n", sum); // Line 9
10    return 0; // Line 10
11 }
```

The debugger console shows the following output:

```
Reading symbols from a.out...
(gdb) break 7
Breakpoint 1 at 0x1151: file main.c, line 7.
(gdb) run
Starting program: /home/a.out
(Thread debugging using libthread_db enabled)
Using host libthread_db library "/nix/store/qhe8ap183mqd04x5jp75981kvy64npr-glibc-2.40-66/lib/libthread_db.so.1".

Breakpoint 1, main () at main.c:7
7       sum = sum + i; // Line 7
(gdb) print i
i = 1
(gdb) display i
1: i = 1
(gdb) display sum
2: sum = 0
(gdb) continue
Continuing.

Breakpoint 1, main () at main.c:7
7       sum = sum + i; // Line 7
1: i = 2
2: sum = 1
```

The second screenshot shows the same GDB online debugger interface. The code is the same as in the first screenshot. The debugger console shows the following output:

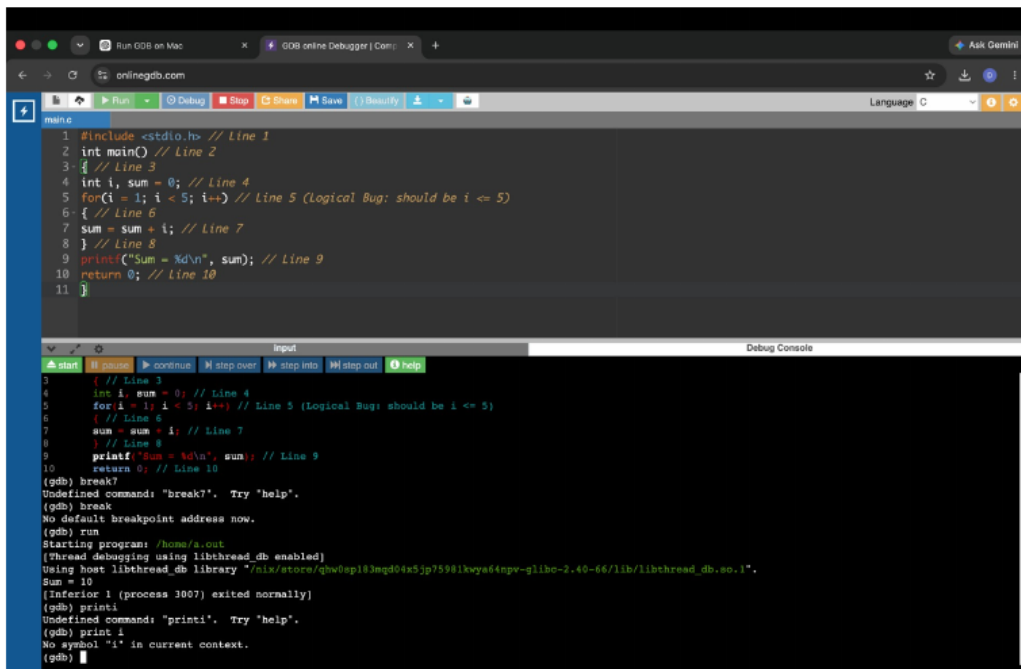
```
Breakpoint 1, main () at main.c:7
7       sum = sum + i; // Line 7
1: i = 2
2: sum = 1
(gdb) continue
Undefined command: "continue". Try 'help'.
(gdb) continue
Continuing.

Breakpoint 1, main () at main.c:7
7       sum = sum + i; // Line 7
1: i = 3
2: sum = 3
(gdb) continue
Continuing.

Breakpoint 1, main () at main.c:7
7       sum = sum + i; // Line 7
1: i = 4
2: sum = 6
(gdb) continue
Continuing.

Sum = 10
```

GDB observation: when i reaches 5, the loop condition becomes false before the body executes. Therefore 5 is omitted from the sum.



```

1 #include <stdio.h> // Line 1
2 int main() // Line 2
3 { // Line 3
4     int i, sum = 0; // Line 4
5     for(i = 1; i < 5; i++) // Line 5 (Logical Bug: should be i <= 5)
6     { // Line 6
7         sum = sum + i; // Line 7
8     } // Line 8
9     printf("Sum = %d\n", sum); // Line 9
10    return 0; // Line 10
11 }

```

Debug Console

```

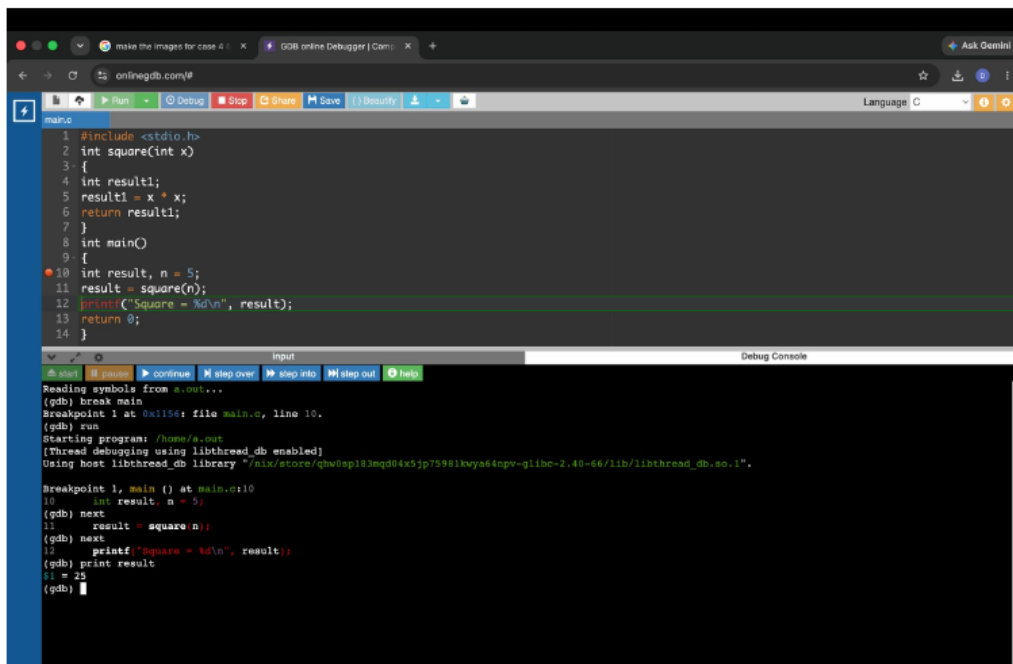
(gdb) break?
Undefined command: "break?". Try 'help'.
(gdb) break
No default breakpoint address now.
(gdb) run
Starting program: /home/a.out
[Thread debugging using libthread_db enabled]
Using host libthread_db library "/nix/store/gbw8p183mqd04x5jp75981kvy64npv-glibc-2.40-66/lib/libthread_db.so.1".
Sum = 10
[Inferior 1 (process 1007) exited normally]
(gdb) print i
Undefined command: "printi". Try 'help'.
(gdb) print i
No symbol "i" in current context.
(gdb)

```

OnlineGDB logical-error debugging session.

Case 5: Debugging Functions (next vs. step)

GDB next executes a function call without entering it, while step enters the called function so its statements and local values can be inspected.



```

1 #include <stdio.h>
2 int square(int x)
3 {
4     int result1;
5     result1 = x * x;
6     return result1;
7 }
8 int main()
9 {
10    int result, n = 5;
11    result = square(n);
12    printf("Square = %d\n", result);
13    return 0;
14 }

```

Debug Console

```

Reading symbols from a.out...
(gdb) break main
Breakpoint 1 at 0x1156: file main.c, line 10.
(gdb) run
Starting program: /home/a.out
[Thread debugging using libthread_db enabled]
Using host libthread_db library "/nix/store/gbw8p183mqd04x5jp75981kvy64npv-glibc-2.40-66/lib/libthread_db.so.1".

Breakpoint 1, main () at main.c:10
10    int result, n = 5;
(gdb) next
11    result = square(n);
(gdb) next
12    printf("Square = %d\n", result);
(gdb) print result
$1 = 25
(gdb)

```

The first screenshot shows the C code in the onlinegdb.com editor:

```

1 #include <stdio.h>
2 int square(int x)
3 {
4     int result1;
5     result1 = x * x;
6     return result1;
7 }
8 int main()
9 {
10    int result, n = 5;
11    result = square(n);
12    printf("Square = %d\n", result);
13    return 0;
14 }
15

```

The debug console shows the following output:

```

(gdb) break main
Breakpoint 1 at 0x1154: file main.c, line 10.
(gdb) run
Starting program: /home/a.out
[Thread debugging using libthread_db enabled]
Using host libthread_db library "/nix/store/qhw0sp183mqd04x5jp7598lkuya64npw-glibc-2.40-66/lib/libthread_db.so.1".

Breakpoint 1, main () at main.c:10
10    int result, n = 5;
(gdb) next
11    result = square(n);
(gdb) step
square (x=5) at main.c:5
5    result1 = x * x;
(gdb) print x
$1 = 5
(gdb) next

```

The second screenshot shows the same code, but with a backtrace displayed in the debug console:

```

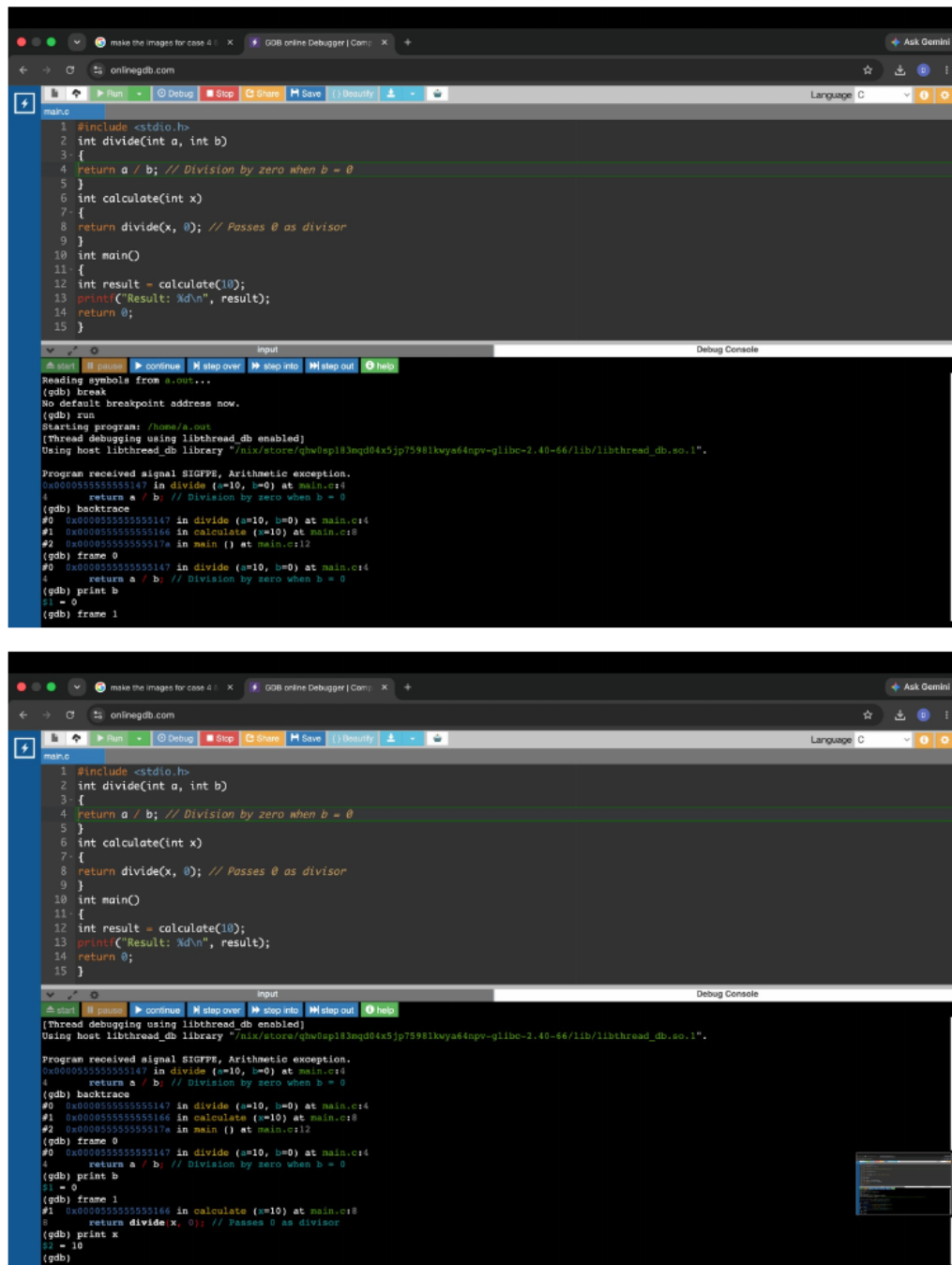
Starting program: /home/a.out
[Thread debugging using libthread_db enabled]
Using host libthread_db library "/nix/store/qhw0sp183mqd04x5jp7598lkuya64npw-glibc-2.40-66/lib/libthread_db.so.1".

Breakpoint 1, main () at main.c:10
10    int result, n = 5;
(gdb) next
11    result = square(n);
(gdb) step
square (x=5) at main.c:5
5    result1 = x * x;
(gdb) print x
$1 = 5
(gdb) next
(gdb) print result1
$2 = 25
(gdb)

```

Case 6: Runtime Error using GDB Backtrace

A division by zero produces SIGFPE. The backtrace command shows the chain divide -> calculate -> main, making it possible to locate the origin of the crash.



Case 7: Array Out-of-Bounds Error using GDB

An array of five integers has valid indices 0 through 4. The condition `i <= 5` also accesses `a[5]`, which is outside the array and causes undefined behavior.

```

1 #include <stdio.h>
2 int main()
3 {
4     int a[5];
5     int i;
6     for(i = 0; i <= 5; i++)
7     {
8         scanf("%d", &a[i]); //line 8
9     }
10    return 0;
11 }

```

Debug Console

```

(gdb) break 8
Breakpoint 1 at 0x114a: file main.c, line 8.
(gdb) run
Starting program: /home/a.out
[Thread debugging using libthread_db enabled]
Using host libthread_db library "/nix/store/gnw0sp183mqd04x5jp7598lkvy64npv-glibc-2.40-66/lib/libthread_db.so.1".

Breakpoint 1, main () at main.c:8
8     scanf("%d", &a[i]); //line 8
(gdb) print &a[0]
Undefined command "print". Try 'help'.
(gdb) print &a[0]
01 = (int *) 0xfffffffffa0
(gdb) print &a[4]
02 = (int *) 0xfffffffff70
(gdb) print &a[5]
03 = (int *) 0xfffffffff74
(gdb)

```

Case 8: Segmentation Fault (SIGSEGV) using GDB

The pointer `p` is NULL (address 0x0). Writing through it with `*p = 10` attempts to access invalid memory, so the program receives SIGSEGV.

```

1 #include <stdio.h>
2 int main()
3 {
4     int *p = NULL; // Pointer points to address 0x0
5     *p = 10; // Attempting to write to address 0x0 causes Segmentation Fault
6     printf("%d\n", *p);
7     return 0;
8 }

```

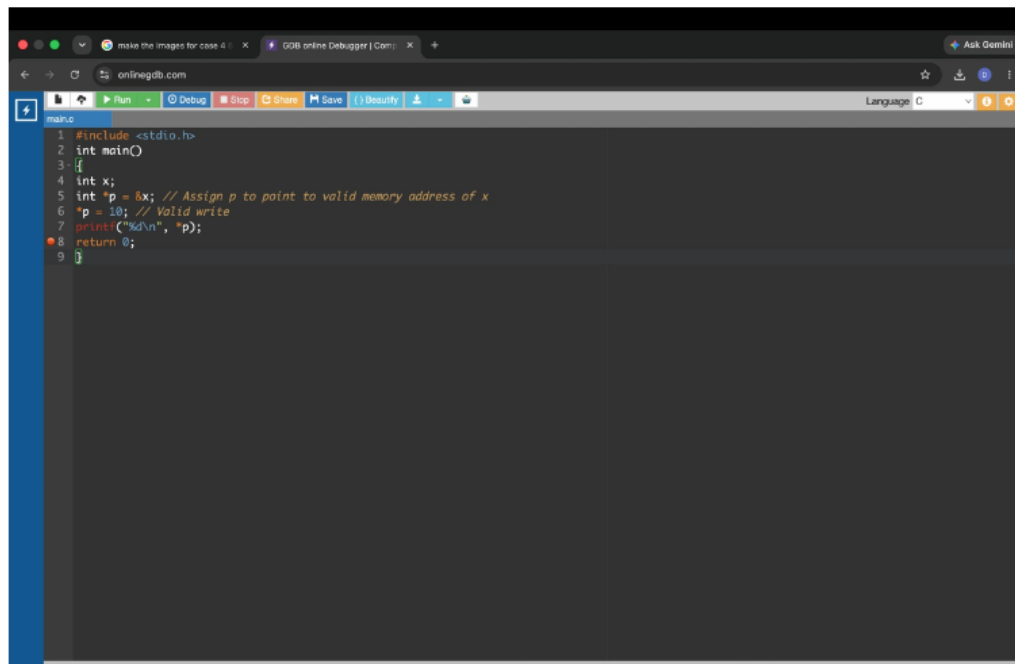
Debug Console

```

(gdb) break 6
Breakpoint 1 at 0x114a: file main.c, line 6.
(gdb) run
Starting program: /home/a.out
[Thread debugging using libthread_db enabled]
Using host libthread_db library "/nix/store/gnw0sp183mqd04x5jp7598lkvy64npv-glibc-2.40-66/lib/libthread_db.so.1".

Program received signal SIGSEGV, Segmentation fault.
0x000055555555514d in main () at main.c:6
6     *p = 10; // Attempting to write to address 0x0 causes Segmentation Fault
(gdb) print p
01 = (int *) 0x0
(gdb) print *p
Cannot access memory at address 0x0
(gdb)

```



The screenshot shows a web browser window with the URL `onlinegdb.com`. The browser's address bar also shows `onlinegdb.com`. The page has a dark theme. At the top, there are tabs for 'make the images for case 4', 'GDB online Debugger | Com...', and 'Ask Gemini'. Below the tabs is a toolbar with buttons for 'Run', 'Debug', 'Stop', 'Share', 'Save', and 'Security'. The main area is a code editor with the following C code:

```
1 #include <stdio.h>
2 int main()
3 {
4     int x;
5     int *p = &x; // Assign p to point to valid memory address of x
6     *p = 10; // Valid write
7     printf("%d\n", *p);
8     return 0;
9 }
```

The code is highlighted with a blue background. The line numbers are on the left. The code is in C, as indicated by the 'Language: C' dropdown in the top right corner of the editor.

Systematic Debugging Strategy

1. Observe expected vs. actual output
2. Reproduce the issue consistently
3. Compile with `-Wall -Wextra -g`
4. Isolate suspicious code
5. Inspect state using `printf` or GDB
6. Apply one minimal fix
7. Recompile and retest
8. Verify edge cases and boundary inputs

Conclusion

Compiler diagnostics, warning flags, `printf` tracing, and GDB provide complementary ways to locate errors. Breakpoints and variable inspection help with logical bugs; next and step help analyze functions; backtrace identifies crash call chains; and pointer/array inspection helps identify invalid memory access.